

Interval Verification Documentation for TrapeziumGPT

July 2026

Abstract

This document describes the architecture, verification logic, functions, and recorded experiments for the **TrapeziumGPT** package. The package verifies upper bounds for the normalized shortest equal-area fence in convex quadrilaterals and also records necessary-condition exclusions for optimality. The search subdivides a four-dimensional rectangle of coordinates for the two free vertices C, D . Each leaf rectangle is either discarded as certainly inadmissible, certified as having a shortest fence value below a threshold θ , certified nonoptimal by the opposite-pair equality condition, or retained as an unresolved survivor. The certificate verifier re-checks both the local leaf claims and the global dyadic tiling of the original domain.

Contents

1	Mathematical Normalization	1
2	Package Architecture	2
3	Rectangle Validation Logic	2
3.1	Cells	2
3.2	Splitting Rule	3
3.3	Leaf Classification	3
3.4	Certificate Semantics	4
4	Auxiliary Certificates	4
4.1	Opposite-Pair Equality	4
4.2	Flat and Small-Area Estimate	4
4.3	Angle Consequence	5
5	Functional Bounds	5
5.1	Vertex Bounds	5
5.2	Opposite-Pair Bounds	5
5.3	Natural and Centered Enclosures	6
6	Function Reference	6
6.1	geom.c/geom.h	6
6.2	series.c/series.h	7
6.3	functionals.c/functionals.h	7
6.4	admissible.c/admissible.h	8
6.5	search.c/search.h	8
6.6	cert.c/cert.h	9
6.7	fence_validate.c	9
6.8	Python Tools and Tests	9

7	Command-Line Interface	10
8	Experiments	11
8.1	Build and Machine	11
8.2	Reference Trapezoid	11
8.3	Threshold $\theta = 1.0496$	11
8.4	Survivor Analysis	12
8.5	Interpretation	13
8.6	Audit Status	13
9	Limitations and Soundness Notes	13

1 Mathematical Normalization

The package studies convex quadrilaterals $ABCD$ with $A = (0, 0)$, $B = (1, 0)$, $C = (c_1, c_2)$, and $D = (d_1, d_2)$. The side AB is assumed to be a longest side and is normalized to length 1. This loses no generality for the target functional, because fence lengths are divided by $\sqrt{\text{Area}(ABCD)}$. A quadrilateral with a longer side can be rescaled and appears in another copy of the normalized search.

The raw search box is

$$c_1 \in [0, 2], \quad c_2 \in [0, 1], \quad d_1 \in [-1, 1], \quad d_2 \in [0, 1].$$

The admissibility predicate keeps only boxes that may contain counterclockwise convex quadrilaterals whose non-base side lengths are at most 1:

$$|BC| \leq 1, \quad |CD| \leq 1, \quad |DA| \leq 1.$$

For a quadrilateral $Q = ABCD$, the code computes six scale-invariant upper bounds $F_i(Q)$ for the normalized shortest fence value:

$$f(Q) \leq \min_{0 \leq i < 6} F_i(Q).$$

The six items are four vertex-angle bounds and two opposite-edge-pair bounds. A box is certified below θ when interval arithmetic proves that the upper endpoint of an enclosure for $\min_i F_i$ is at most θ . Consequently, certified boxes are one-sided exclusions for the problem $f(Q) > \theta$: they prove that no admissible quadrilateral in the box has normalized shortest fence value above the threshold.

2 Package Architecture

The C implementation is arranged in layers:

geom.c, geom.h Arb arithmetic wrappers, four-variable forward-mode automatic differentiation jets, and basic point/vector operations.

series.c, series.h Rigorous enclosures for $g(\gamma) = \gamma / \sin \gamma$ and $g'(\gamma)$, used by the parallel-safe opposite-pair formula near $\gamma = 0$.

functionals.c, functionals.h Construction of the six upper-bound functionals, natural and centered interval enclosures, the minimum envelope used by certification, and the optional `functionals\_opposite\_pairs\_disjoint` nonoptimality certificate.

`admissible.c`, `admissible.h` Conservative interval tests that discard boxes containing no admissible normalized quadrilateral, plus the flat/small-area certificate `box\_small\_area\_certifies\_1`.

`search.c`, `search.h` Dyadic subdivision engine. It classifies boxes as discarded, certified low, flat-area low, nonoptimal, or surviving; splits unresolved boxes; collects statistics; and can seed a refinement run from previous survivors.

`cert.c`, `cert.h` JSONL certificate writer and stateless certificate verifier. The verifier recomputes local claims and checks that all leaves tile the original domain by the same dyadic splitting rule.

`fence_validate.c` Command-line front end for searching, refining, verifying, and evaluating a point.

`analyze_cert.py` Safe postprocessing of certificate files. It does not replace certificate verification.

`tests/test_trap.c` Unit tests for the reference trapezoid, pair-bound formula agreement, series evaluation, inadmissibility, one interior certification, the pair-equality certificate, and the flat-area certificate.

3 Rectangle Validation Logic

3.1 Cells

A search cell is a four-dimensional rectangle

$$X = [c_1^-, c_1^+] \times [c_2^-, c_2^+] \times [d_1^-, d_1^+] \times [d_2^-, d_2^+].$$

Equivalently, it is a product of two planar rectangles:

$$X_C = [c_1^-, c_1^+] \times [c_2^-, c_2^+], \quad X_D = [d_1^-, d_1^+] \times [d_2^-, d_2^+].$$

The initial cell has

$$X_C = [0, 2] \times [0, 1], \quad X_D = [-1, 1] \times [0, 1].$$

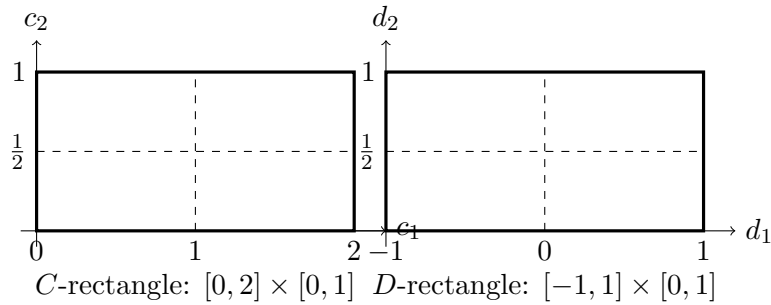


Figure 1: Equal-axis initial planar grids for C and D . Combining one rectangle from the C -grid with one rectangle from the D -grid gives a four-dimensional cell.

3.2 Splitting Rule

The root spans are

$$(2, 1, 2, 1)$$

for the coordinates (c_1, c_2, d_1, d_2) . The search compares scaled widths

$$\frac{c_1^+ - c_1^-}{2}, \quad c_2^+ - c_2^-, \quad \frac{d_1^+ - d_1^-}{2}, \quad d_2^+ - d_2^-.$$

The coordinate with largest scaled width is bisected. If the coordinate is c_1 or c_2 , only the C -rectangle changes; if it is d_1 or d_2 , only the D -rectangle changes. This rule is deterministic and is used by both the search and the certificate verifier.

3.3 Leaf Classification

For a box X , the search applies the following logic:

1. If `box\_certainly\_inadmissible` proves that the entire box violates convexity, side-length, or optional half-symmetry constraints, the leaf is written with status **discarded**.
2. If `--flat-area-cert` is enabled and `box\_small\_area\_certifies\_low` proves the flat/small-area estimate below, the leaf is written with status `flat\_area`. This is a genuine low-fence certificate.
3. If `--pair-eq-cert` is enabled and `functionals\_opposite\_pairs\_disjoint` proves that the two opposite-pair fence intervals are disjoint, the leaf is written with status **nonoptimal**. This excludes an optimizer by a necessary optimality condition, but is not by itself a low-fence claim.
4. Otherwise, `functionals\_min` computes an interval enclosure $[f_{\text{lo}}, f_{\text{hi}}]$ for the minimum of the six upper-bound items. If the enclosure is finite and $f_{\text{hi}} \leq \theta$, the leaf is written with status **certified**.
5. If the box is not certified and its largest scaled width is at most `wfloor`, it is written with status **survivor**. A survivor is not evidence of optimality; it is merely an unresolved part of the outer neighborhood.
6. Otherwise the box is split into two children and the process continues.

The search can be restarted from survivors by passing `--refine FILE`. The refined output plus the discarded/certified leaves of the previous run forms a complete certificate for the original root box.

3.4 Certificate Semantics

A complete verified certificate proves:

every admissible quadrilateral outside the survivor union is either
certified below θ or certified nonoptimal.

Statuses **certified** and `flat\_area` are low-fence certificates: they prove $f(Q) \leq \theta$ throughout the box. Status **nonoptimal** means that no optimizer lies in the box, because a necessary optimality condition fails. It does not assert a pointwise low-fence bound.

Therefore a certificate with `--pair-eq-cert` enabled should be read as an exclusion certificate for possible above-threshold optimizers, not simply as a partition into low boxes and survivors. It does not prove that survivor boxes contain values above θ , nor does it produce a lower bound for the true optimum. It gives an outer neighborhood for currently unresolved candidate configurations.

4 Auxiliary Certificates

4.1 Opposite-Pair Equality

The two opposite-pair items correspond to the fence constructions associated to the pairs (AB, CD) and (BC, DA) . A theoretical necessary condition for optimality is that these two fences agree at an optimizer. The optional certificate `--pair-eq-cert` evaluates interval enclosures for the two opposite-pair items. If the finite intervals are disjoint, then no point in the box can satisfy the equality condition, and the box is written with status `nonoptimal`.

This certificate is independent of the threshold θ . It removes regions that cannot contain a maximizing quadrilateral even when the current upper-bound envelope is too wide to certify $f(Q) \leq \theta$.

4.2 Flat and Small-Area Estimate

Let

$$A = \text{Area}(ABCD), \quad h = \max\{c_2, d_2\}.$$

For the fence associated to the pair of opposite sides (AB, CD) , a direct construction gives

$$L_{AB,CD} \leq h.$$

Indeed, the relevant separating curve can be chosen with length no larger than the largest height of the two top vertices over the base line AB .

For an admissible counterclockwise convex quadrilateral in the normalized domain,

$$A = \text{Area}(ABC) + \text{Area}(ACD) > \frac{c_2}{2}, \quad A = \text{Area}(ABD) + \text{Area}(BCD) > \frac{d_2}{2}.$$

Hence $h \leq 2A$, and therefore

$$\frac{L_{AB,CD}}{\sqrt{A}} \leq \frac{h}{\sqrt{A}} \leq 2\sqrt{A}.$$

Consequently, if interval arithmetic proves

$$A_{\text{ub}} \leq \frac{\theta^2}{4},$$

then the whole box has normalized shortest fence value at most θ . This is implemented by `box\_small\_area\_certifies\_low` and recorded with status `flat\_area`.

For the threshold used below,

$$\theta = 1.0496, \quad \theta^2 = 1.10166016, \quad \frac{\theta^2}{4} = 0.27541504.$$

4.3 Angle Consequence

The vertex fence at an angle α has normalized upper bound $\sqrt{\alpha}$. Thus, if the smallest angle of $ABCD$ is less than θ^2 , the quadrilateral is already below threshold. Any possible above-threshold optimizer must therefore have all four angles at least θ^2 , so its largest angle is at most

$$2\pi - 3\theta^2.$$

For $\theta = 1.0496$, this is

$$2.978204827 \text{ radians} = 170.638567^\circ,$$

or equivalently the gap from a straight angle is at least

$$3\theta^2 - \pi = 0.163387826 \text{ radians}.$$

This angle observation explains why very flat quadrilaterals should not be optimal. The implemented small-area certificate above is the direct quantitative exclusion used in the current computation.

5 Functional Bounds

5.1 Vertex Bounds

For a vertex V with neighboring vertices P, Q , the vertex item is

$$F_V = \sqrt{\angle PVQ}.$$

The angle is computed as

$$\text{atan2}(|(P - V) \times (Q - V)|, (P - V) \cdot (Q - V)).$$

If a coarse interval contains a degenerate vertex case where `atan2` becomes indeterminate, the value interval is clamped conservatively to $[0, \pi]$, and derivative information is treated as indeterminate. In centered mode this forces a conservative fallback to the natural enclosure.

5.2 Opposite-Pair Bounds

There are two opposite-pair items: (AB, CD) and (BC, DA) . Let the two supporting line directions make acute angle $\gamma \in [0, \pi/2]$, and let $A_Q = \text{Area}(ABCD)$. The implementation uses two algebraically identical formulas.

When $\inf \gamma > \text{GAMMA_SPLIT}$, the apex form is used. Let O be the intersection point of the two supporting lines, and let T_0, T_1 be the areas of the two triangles formed by the non-pair edges and O . Then

$$F_{\text{pair}}^2 = \frac{\gamma(T_0 + T_1)}{A_Q}.$$

Near parallelism, the symmetric formula is used:

$$F_{\text{pair}}^2 = \frac{\gamma}{\sin \gamma} \frac{p_a + p_b}{2A_Q},$$

where each p is a product of distances from the endpoints of one non-pair edge to the two opposite supporting lines. This form avoids the unstable intersection point O when γ is small.

5.3 Natural and Centered Enclosures

The natural enclosure evaluates each expression directly over interval boxes. The centered enclosure uses a mean-value form

$$F_i(m) + \sum_{k=0}^3 \partial_k F_i(X) [-r_k, r_k],$$

where m is the coordinate midpoint and r_k is half the coordinate width. The derivatives $\partial_k F_i(X)$ are obtained by forward-mode automatic differentiation over Arb balls. In centered mode the code combines the centered and natural enclosures conservatively: if both are finite and overlap, their intersection is used; if they are finite but disjoint, their hull is used; if one is non-finite, the finite one is used.

6 Function Reference

This section lists the implemented functions by module. Static functions are included because they encode important proof logic.

6.1 geom.c/geom.h

Function	Purpose
<code>jet_init</code>	Initializes the value ball and all four derivative balls of a jet.
<code>jet_clear</code>	Clears all Arb storage owned by a jet.
<code>jet_set</code>	Copies a jet value and its derivative enclosures.
<code>jet_set_var</code>	Seeds one moduli coordinate as an automatic-differentiation variable with derivative 1 in coordinate k .
<code>jet_set_const</code>	Sets a constant Arb value and zero derivatives.
<code>jet_set_si</code>	Sets a signed integer constant and zero derivatives.
<code>jet_add</code>	Adds two jets componentwise.
<code>jet_sub</code>	Subtracts two jets componentwise.
<code>jet_neg</code>	Negates value and derivatives.
<code>jet_mul</code>	Multiplies jets using the product rule; temporaries allow aliasing.
<code>jet_div</code>	Divides jets using the quotient rule; non-finite balls are preserved when the denominator straddles zero.
<code>jet_sqr</code>	Squares a jet and applies $d(a^2) = 2a da$.
<code>jet_sqrt</code>	Applies square root with derivative $da/(2\sqrt{a})$. Degenerate input may produce non-finite derivative enclosures, which higher layers handle conservatively.
<code>jet_abs</code>	Computes $ a $. If the value straddles zero, the value becomes $[0, \sup a]$ and derivatives receive sign uncertainty.
<code>jet_atan2</code>	Computes $\text{atan2}(y, x)$ and its derivative $(x dy - y dx)/(x^2 + y^2)$.
<code>jet_mul_si</code>	Multiplies a jet by a signed integer.
<code>jet_mul_arb</code>	Multiplies a jet by a scalar Arb constant.
<code>jpt_init</code>	Initializes a two-coordinate jet point.
<code>jpt_clear</code>	Clears a jet point.
<code>jpt_sub</code>	Subtracts two jet points.
<code>jet_cross</code>	Computes the two-dimensional cross product of two jet vectors.
<code>jet_dot</code>	Computes the dot product of two jet vectors.
<code>trap_cross</code>	Scalar Arb cross product for raw vector components.
<code>trap_dot</code>	Scalar Arb dot product for raw vector components.

6.2 series.c/series.h

Function	Purpose
<code>series_build</code>	Static initializer that computes Taylor coefficients for $\gamma/\sin \gamma$ as rational numbers and stores them as high-precision Arb balls.
<code>ensure_coeffs</code>	Static guard that lazily initializes the coefficient table; protected by an OpenMP critical section when OpenMP is enabled.
<code>series_cleanup</code>	Clears cached coefficient balls. Called by the CLI and tests before exit.

Function	Purpose
<code>ghi_ball</code>	Static helper that turns the upper endpoint of a γ interval into a thin Arb ball.
<code>clamp_nonneg</code>	Static helper that intersects γ with $[0, \pi/2]$, valid because the pair angle is acute by construction.
<code>gamma_over_sin_series</code>	Taylor-series enclosure for $g(\gamma) = \gamma/\sin \gamma$, with a rigorous positive tail bound.
<code>gamma_over_sin_value</code>	Monotone endpoint enclosure for $g(\gamma)$, avoiding the removable $0/0$ singularity.
<code>gamma_over_sin_deriv</code>	Encloses $g'(\gamma)$ by direct formula away from zero and by series near zero.
<code>jet_gamma_over_sin</code>	Applies g to a jet using the value enclosure and derivative enclosure.

6.3 functionals.c/functionals.h

Function	Purpose
<code>vbound</code>	Static helper for the vertex bound $\sqrt{\angle PVQ}$.
<code>pdist</code>	Static helper for the distance from a point to a line, used in the parallel-safe pair formula.
<code>tri_area</code>	Static helper for triangle area $\frac{1}{2} (Q - P) \times (O - P) $.
<code>pair_bound</code>	Static helper implementing the opposite-pair bound. It selects the apex formula or the parallel-safe formula unless a test forces one.
<code>compute_items</code>	Static builder for the six item jets from the four coordinate jets. It also constructs the quadrilateral area $A_Q = (c_2 + c_1d_2 - d_1c_2)/2$.
<code>functionals_eval</code>	Public routine that encloses all six items on a coordinate box, in natural or centered form.
<code>functionals_pair_forced</code>	Test hook that evaluates one pair item at a point while forcing the apex or parallel-safe formula.
<code>functionals_min</code>	Encloses the minimum envelope over the six items. Non-finite item enclosures are ignored; if all six are non-finite, the result is $+\infty$, so the search cannot certify the box.
<code>functionals_opposite_pairs_disjoint</code>	Encloses the two opposite-pair items and returns true only when both finite intervals are disjoint, certifying failure of the pair-equality necessary condition for optimality.

6.4 admissible.c/admissible.h

Function	Purpose
<code>set_box</code>	Static helper converting double endpoints into an Arb interval.
<code>sqrlen</code>	Static helper computing squared vector length in Arb arithmetic.

Function	Purpose
<code>box_certainly_inadmissible</code>	Returns true only when a whole box is proved inadmissible. It checks CCW convexity through cross products, side lengths through squared distances, and optionally the half-domain condition $c_1 + d_1 \geq 1$.
<code>box_small_area_certifies_low</code>	Returns true only when the interval upper bound for area satisfies $A_{ub} \leq \theta^2/4$, so the estimate $L_{AB,CD}/\sqrt{A} \leq 2\sqrt{A}$ proves the box below threshold.

6.5 search.c/search.h

Function	Purpose
<code>search_root_box</code>	Writes the full coordinate domain $[0, 2] \times [0, 1] \times [-1, 1] \times [0, 1]$.
<code>st_init</code>	Static stack initializer for pending boxes.
<code>st_free</code>	Static stack destructor.
<code>st_push</code>	Static stack push with geometric reallocation.
<code>st_pop</code>	Static stack pop; returns false when empty.
<code>widest_coord</code>	Static helper selecting the coordinate with largest root-span-scaled width.
<code>classify</code>	Static core classifier. It applies inadmissibility, optional flat-area certification, optional pair-equality nonoptimality, functional certification, optional adaptive precision, survivor flooring, and child generation.
<code>stats_add_leaf</code>	Static statistics accumulator for a final leaf.
<code>run_serial</code>	Static depth-first serial driver over a seeded stack.
<code>run_parallel</code>	Static OpenMP driver using a shared stack and critical sections for stack/statistics updates.
<code>run_stack</code>	Static dispatcher that initializes statistics and selects serial or parallel execution.
<code>search_run</code>	Starts a search from the root box.
<code>search_run_seeded</code>	Starts a search from explicit seed boxes, used by <code>--refine</code> .

6.6 cert.c/cert.h

Function	Purpose
<code>status_name</code>	Static conversion from leaf status enum to JSON status string.
<code>cert_write_leaf</code>	Leaf callback that writes one JSON line containing box, status, active item, enclosure, and precision.
<code>parse_line</code>	Static parser for the simple JSONL certificate format.
<code>parse_status</code>	Static parser for status strings; accepts <code>low</code> as an alias for <code>certified</code> , and recognizes <code>flat_area</code> and <code>nonoptimal</code> .
<code>cmp_double</code>	Static total ordering helper for doubles used in sorting boxes.
<code>cmp_box_arrays</code>	Static lexicographic comparison of box endpoints.
<code>leaf_cmp</code>	Static comparator for sorting certificate leaves.

Function	Purpose
<code>find_leaf</code>	Static binary search for an exact dyadic box in the sorted certificate array.
<code>widest_coord_cert</code>	Static copy of the deterministic splitting rule used by the verifier.
<code>verify_leaf_claim</code>	Static local audit of one leaf: discarded boxes must still be certainly inadmissible; certified and flat-area boxes must recompute below θ ; nonoptimal boxes must recompute disjoint opposite-pair intervals; survivors require no value claim.
<code>verify_cover_node</code>	Static recursive tiling audit. It descends the dyadic tree and requires every generated node to be represented by exactly one certificate leaf or by its two children.
<code>cert_load_survivors</code>	Loads survivor boxes from a certificate file for a refinement run.
<code>cert_verify</code>	Public verifier combining parsing, duplicate detection, global coverage checking, and local claim recomputation.

6.7 `fence\_validate.c`

Function	Purpose
<code>usage</code>	Static helper printing command-line options.
<code>main</code>	Parses options, dispatches point evaluation, verification, full search, or survivor refinement; writes certificates and prints statistics.

6.8 Python Tools and Tests

Function or file	Purpose
<code>analyze_cert.py</code>	Summarizes certificates safely. It reports survivor volume and separates certainly-admissible survivor cores from boundary-straddling survivors.
<code>tests/test_trap.c: ok</code>	Records a unit-test pass/fail result.
<code>tests/test_trap.c: ub, lb</code>	Convert Arb upper and lower endpoints to doubles for tests.
<code>tests/test_trap.c: approx</code>	Checks that an Arb enclosure lies within a tolerance of a reference value.
<code>tests/test_trap.c: test_Tstar</code>	Checks the reference trapezoid values and active pair item.
<code>tests/test_trap.c: test_pair_forms</code>	Checks agreement of the two opposite-pair formulas.
<code>tests/test_trap.c: test_series</code>	Checks $\gamma/\sin \gamma$ series enclosures against direct Arb evaluation.
<code>tests/test_trap.c: test_inadmissible</code>	Checks conservative inadmissibility on known examples.
<code>tests/test_trap.c: test_interior_certify</code>	Checks that an interior sub-threshold box certifies.

Function or file	Purpose
<code>tests/test_trap.c:</code> <code>test_pair_equality_certificate</code>	Checks that a box with disjoint opposite-pair intervals is certified nonoptimal, while a small box around T^* is not rejected by this certificate.
<code>tests/test_trap.c:</code> <code>test_flat_area_certificate</code>	Checks that small-area boxes are certified low and that a small box around T^* is not eliminated by the flat-area estimate.

7 Command-Line Interface

The executable is `fence\_validate`. The principal options are:

- `--theta T` Threshold for certification.
- `--wfloor W` Scaled-width floor at which unresolved boxes become survivors.
- `--prec P` Starting Arb precision in bits.
- `--maxprec M` Cap for adaptive precision.
- `--form natural|centered` Interval enclosure mode.
- `--half` Use the optional reflection half-domain $c_1 + d_1 \geq 1$.
- `--serial` Force single-threaded execution.
- `--adaptive` Retry marginal unresolved boxes at higher precision.
- `--pair-eq-cert` Certify boxes as `nonoptimal` when the two opposite-pair fence intervals are disjoint.
- `--flat-area-cert` Certify boxes as `flat\_area` when the small-area estimate proves normalized fence value below θ .
- `--max-leaves N` Debugging cap. Output from capped runs should not be treated as a complete certificate.
- `--out FILE` Write a JSONL certificate stream.
- `--stats` Print summary statistics.
- `--verify FILE` Re-check a certificate.
- `--eval c1 c2 d1 d2` Print the six items and minimum at a point.
- `--refine FILE` Seed the search from survivor boxes in a previous certificate.

8 Experiments

8.1 Build and Machine

The current threshold experiment used serial execution with centered enclosures, precision 70, and both auxiliary certificates:

```
make OMP=0
```

Machine characteristics for the timings:

Host	beni-ThinkPad-P53
OS/kernel	Ubuntu 24.04, Linux 6.11.0-29-generic
CPU	Intel Core i7-9750H at 2.60 GHz, 6 cores / 12 threads
Memory	31 GiB RAM, 29 GiB swap
Compiler	Ubuntu GCC 13.3.0
Mode	make OMP=0, --serial

8.2 Reference Trapezoid

The reference trapezoid used in diagnostics is approximately

$$C^* = (0.6417451566, 0.7071006812), \quad D^* = (0.3582548434, 0.7071006812).$$

Point evaluation gives

$$f(T^*) \approx 1.04968581549,$$

with an opposite-pair item active.

8.3 Threshold $\theta = 1.0496$

The following sequence refines only the survivors of the previous run. Each run uses `--theta 1.0496`, `--prec 70`, `--form centered`, `--serial`, `--flat-area-cert`, and `--pair-eq-cert`.

Run	wfloor	Time (s)	Leaves	Discarded	Certified	Flat	Nonopt.	Survivors	Survivor volume
Base	0.025	82.416	215758	11555	116319	213	17884	69787	1.66385×10^{-2}
Refine 1	0.0125	112.479	360798	2967	282735	0	50065	25031	3.72991×10^{-4}
Refine 2	0.00625	35.136	119594	28	51386	0	55878	12302	1.14571×10^{-5}
Refine 3	0.003125	18.052	66134	0	17312	0	39816	9006	5.24218×10^{-7}

The commands were:

```
./fence_validate --theta 1.0496 --wfloor 0.025 --prec 70 \
--form centered --serial --flat-area-cert --pair-eq-cert \
--out flat_pair_cert_w0025.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.0125 --prec 70 \
--form centered --serial --flat-area-cert --pair-eq-cert \
--refine flat_pair_cert_w0025.jsonl \
--out flat_pair_refine_w00125.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.00625 --prec 70 \
--form centered --serial --flat-area-cert --pair-eq-cert \
--refine flat_pair_refine_w00125.jsonl \
--out flat_pair_refine_w000625.jsonl --stats

./fence_validate --theta 1.0496 --wfloor 0.003125 --prec 70 \
--form centered --serial --flat-area-cert --pair-eq-cert \
--refine flat_pair_refine_w000625.jsonl \
--out flat_pair_refine_w0003125.jsonl --stats
```

8.4 Survivor Analysis

The postprocessor applied to the last refinement file reports that all remaining survivors are certainly admissible, with zero boundary-straddling survivor volume. The survivor union is contained in the product

$$R_C \times R_D,$$

where

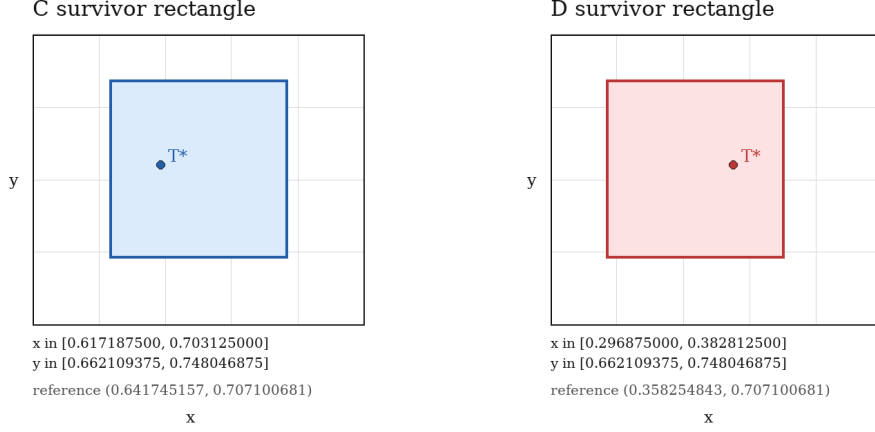
$$R_C = [0.6171875, 0.703125] \times [0.662109375, 0.748046875],$$

$$R_D = [0.296875, 0.3828125] \times [0.662109375, 0.748046875].$$

The survivor volume bound is

$$5.24218 \times 10^{-7},$$

and the farthest point of a survivor box from T^* , in the coordinate box metric used by the analysis script, is 0.07065.



Current certificate: theta = 1.0496, wfloor = 0.003125, survivors = 9006, volume <= 5.24218e-7
All remaining survivors are certainly admissible; boundary-straddling survivor volume is 0.

Figure 2: Equal-axis PNG export of the current survivor rectangles for C and D , together with the reference trapezoid point.

8.5 Interpretation

For $\theta = 1.0496$, the current refinement sequence leaves only boxes inside $R_C \times R_D$. Outside this product rectangle, every admissible normalized quadrilateral is either certified to have shortest fence value at most 1.0496 or certified nonoptimal by the opposite-pair equality condition. Thus there is no possible above-threshold optimizer outside $R_C \times R_D$.

This is stronger than the earlier run without auxiliary certificates, but it must be read with the correct status semantics: **nonoptimal** is not a low-fence estimate. It is a separate exclusion using a necessary condition at optimality.

8.6 Audit Status

The listed files are incremental refinement files: each refinement replaces the survivor leaves of the preceding run. To obtain a single auditable certificate for the whole root domain, one should assemble the chain by replacing survivors at each stage, then run `./fence\_validate --verify` with the same certificate options and sufficient precision. The recorded statistics above are the local statistics of the refinement steps and the postprocessed survivor analysis.

9 Limitations and Soundness Notes

- Certification is based on upper bounds for the shortest fence. A certified box proves “below threshold”, while a survivor only means that the current upper-bound enclosures were not tight enough to certify.
- Status `nonoptimal` is not an upper-bound statement. It certifies failure of the opposite-pair equality necessary condition for an optimizer. Conclusions that use `nonoptimal` boxes should be stated as exclusion of possible optimizers, not as pointwise low-fence bounds.
- Status `flat\_area` is an upper-bound statement. It uses $L_{AB,CD}/\sqrt{A} \leq 2\sqrt{A}$ and $A_{ub} \leq \theta^2/4$ to prove the box below threshold.
- Coarse boxes near degeneracies can have very wide interval enclosures. This is intentional: non-finite or loose enclosures force splitting or survival, never false certification.
- The JSONL enclosure printed in a certificate is diagnostic. The verifier ignores the recorded enclosure and recomputes claims from the box, threshold, form, half-domain flag, and precision.
- A run stopped by `--max-leaves` is useful for debugging but is not a complete certificate and should fail the full coverage check.
- Parallel execution uses a shared stack, so output order may vary. The verifier sorts leaves and checks dyadic coverage, so certificate correctness does not depend on output order.